ClinicFlow

A clinic appointment system taken from a laptop to a monitored Kubernetes cluster on AWS, then torn back down with proof that nothing was left behind.

FASTAPI + REACT / DOCKER + TRIVY / TERRAFORM + EKS

HELM + ARGO CD / PROMETHEUS + GRAFANA / GITHUB ACTIONS

01 THE RULE
ONE DOCTOR, ONE MOMENT

02 THE BUILD
TWO IMAGES, NON ROOT

03 THE PIPELINE
SCAN BEFORE PUSH

04 THE CLUSTER
TERRAFORM, HELM, ARGO

05 THE PROOF
FAULTS, METRICS, TEARDOWN

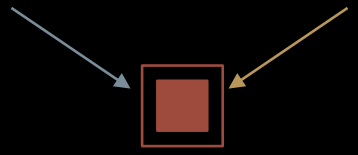
15	16	2	4	0
REST ENDPOINTS	TESTS, ALL GREEN	IMAGES, SCANNED	FAULTS INJECTED	RESOURCES LEFT RUNNING

SASWATA DAS 24BCS10248

One rule in the domain decided every engineering choice that follows.

01

THE RULE



A doctor cannot be in two places at one moment.

So correctness lives in the database, not in the handler.

SLIDES 3 TO 4

02

THE BUILD



Two images, non root, 88.7 MB and 26.7 MB.

So what runs in production is what the tests ran on.

SLIDES 5 TO 6

03

THE PIPELINE



Test, build, scan, then push. In that order.

So an image that fails never reaches the registry.

SLIDES 7 TO 9

04

THE CLUSTER



Terraform builds it, Argo CD keeps it that way.

So no credential for the cluster is held outside it.

SLIDES 10 TO 11

05

THE PROOF



Faults injected, tails measured, account swept.

So every claim here has an artefact behind it.

SLIDES 12 TO 15

Read left to right, these five columns are the whole project.

A clinic schedules people, so two bookings can collide on one doctor.

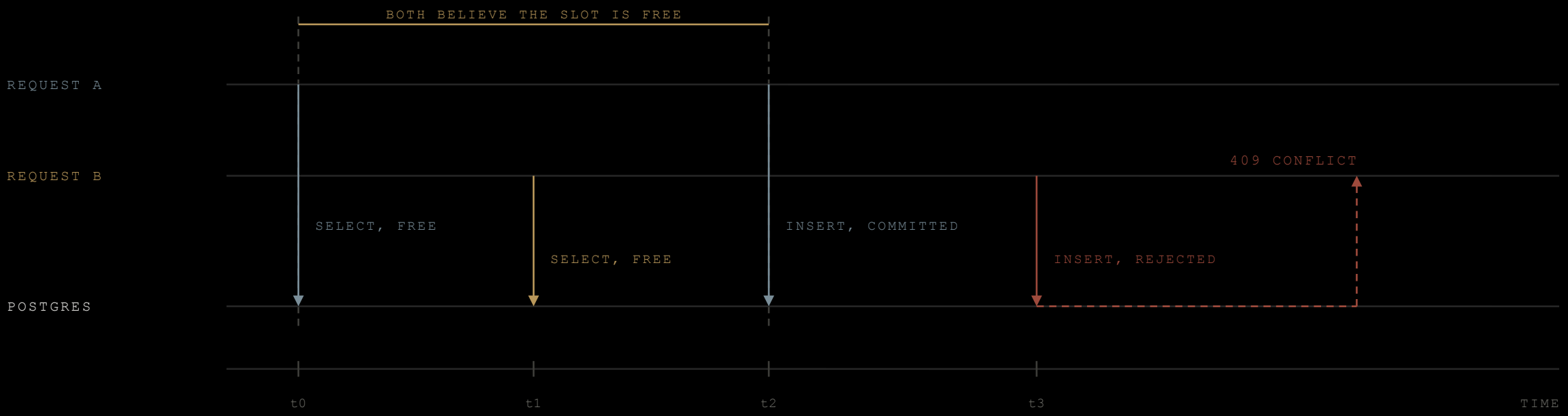


The pair in brass must never repeat while an appointment is live.

ONE MORNING, THREE DOCTORS



Two requests can both pass a free slot check before either one inserts.



THE FIX, ONE LAYER DOWN

```
CREATE UNIQUE INDEX uq_doctor_slot_live
  ON appointments (doctor_id, scheduled_at)
  WHERE status IN ('scheduled', 'completed');
```

Partial, so cancelling frees the slot.

The second insert raises IntegrityError. The API answers 409, and no retry loop is needed.

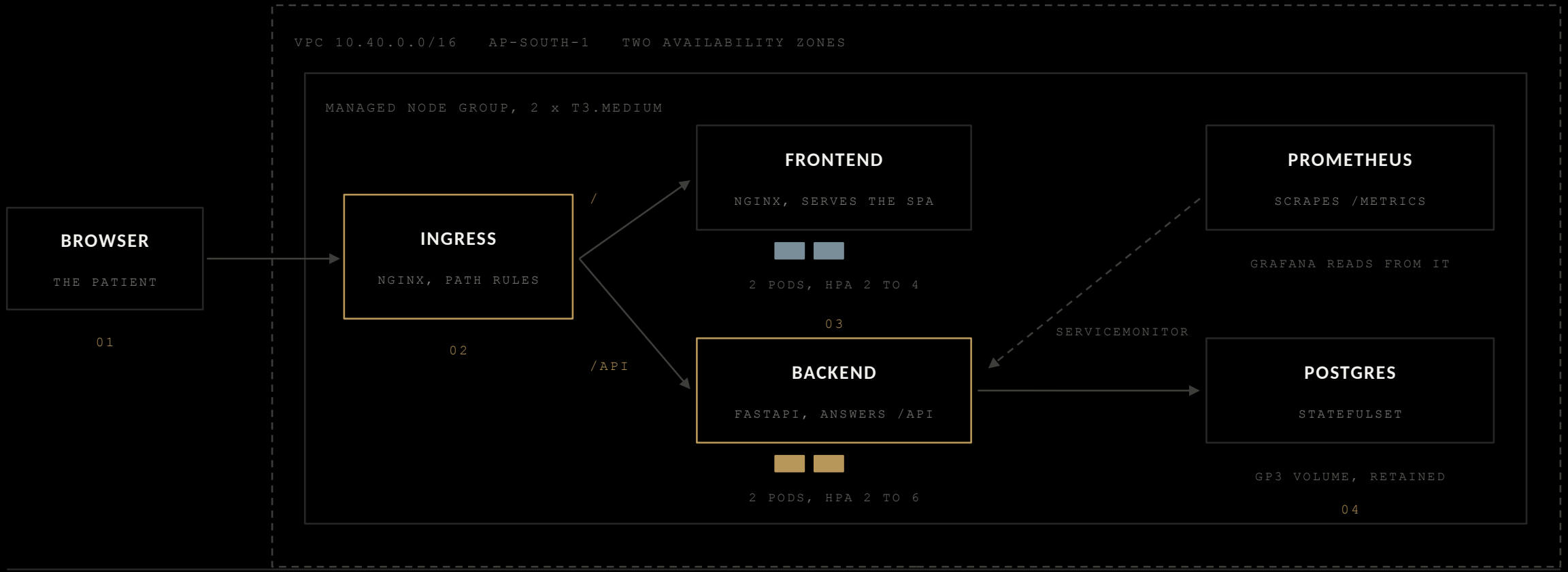
Checking first cannot close the window. Only the database sees both requests.

Each layer was chosen so the layer above it could be verified.

PYTHON / JAVASCRIPT / SQL / HCL / YAML / BASH

LAYER	WHAT IT IS	HOW IT IS VERIFIED
INTERFACE	React 18, Vite 6, no component library, 49 KB gzipped	*the browser, at phone width*
API	FastAPI, Pydantic schemas, 15 endpoints	*16 pytest cases*
PERSISTENCE	SQLAlchemy 2.0, Alembic migrations	*a migration that runs forwards*
DATA	PostgreSQL 16, partial unique index	*a concurrent double booking*
IMAGE	Docker multi stage, non root, read only root filesystem	*Trivy, HIGH and CRITICAL*
ORCHESTRATION	Kubernetes 1.31 on EKS, Helm 3, Argo CD	*kubectl, against the live cluster*
INFRASTRUCTURE	Terraform, VPC, managed node group, EBS CSI	*the AWS API, not the state file*
TELEMETRY	Prometheus histograms, Grafana, ServiceMonitor	*a burst, visible in the tail*

One request's path through the cluster names every component in it.

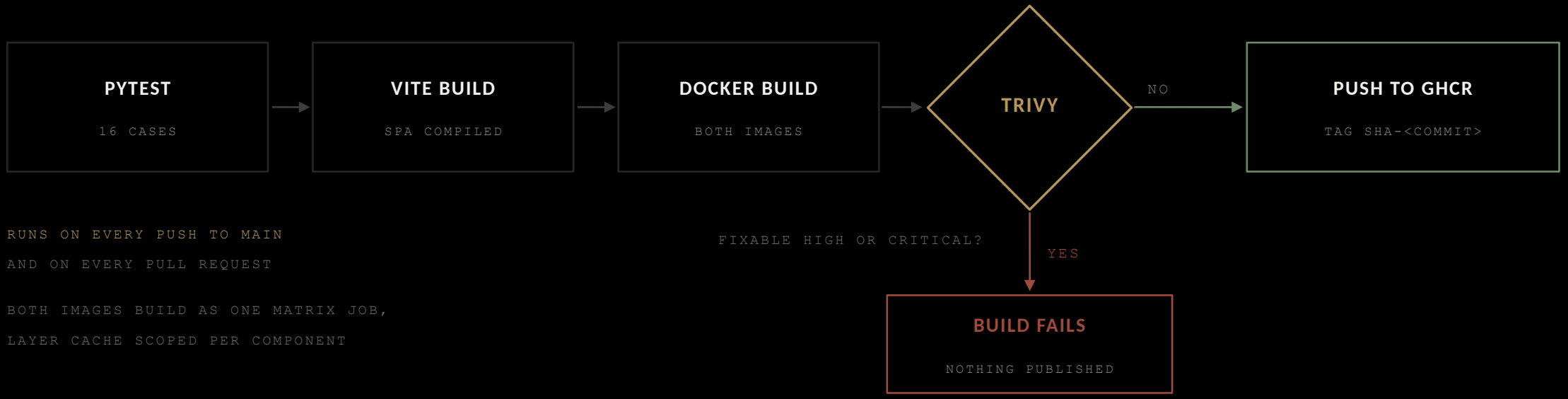


ANTI AFFINITY
replicas never share a node

READ ONLY ROOT
both images run as a non root user

ONE VOLUME
the database is the only thing that cannot scale sideways

The security gate sits between build and push, not after it.



RUNS ON EVERY PUSH TO MAIN
AND ON EVERY PULL REQUEST

BOTH IMAGES BUILD AS ONE MATRIX JOB,
LAYER CACHE SCOPED PER COMPONENT

WHAT EACH JOB HANDS TO THE NEXT

PYTEST	VITE	BUILDX	TRIVY	GHCR
a junit report	a dist/ bundle	two image layers, cached	a json report, kept 30 days	an immutable digest

WHY THIS ORDER
the push job depends on the scan job

WHY SHA TAGS
latest is not a version you can roll back to

WHY UNFIXED CVES PASS
a gate nobody can act on gets ignored

On its first run the gate rejected both images, which is how I know.

42	2	3	0
FIXABLE, FRONTEND BASE IMAGE	OF THEM CRITICAL	PINS THAT HAD TO MOVE	IMAGES PUBLISHED THAT DAY
			NONE

BAR LENGTH IS THE FINDING COUNT, TO SCALE

WHERE THE 42 CAME FROM, AND NONE OF IT WAS MY CODE

BASE TAG PINNED
NGINX-UNPRIVILEGED:1.27-ALPINE

→

NEVER REBUILT
UPSTREAM MOVED ON WITHOUT IT

→

ALPINE PACKAGES STALE
OPENSSL, BUSYBOX, LIBCRYPTO

→

42 FINDINGS
NONE OF IT MY CODE

Pinning a base image pins its vulnerabilities too.

AND THE ONE FINDING THAT WAS MINE

STARLETTE
CVE-2025-62727

→

BUMP FORCED FASTAPI UP
0.115 TO 0.143

→

WHICH BROKE THE EXPORTER
INSTRUMENTATOR 7.0.2

→

THREE PINS, ONE COMMIT
GREEN AGAIN

A gate that has never rejected anything has not been tested.

CI holds no cluster credential, because the cluster pulls instead.



THE PUSH MODEL, NOT USED



A runner holds a credential that can change production. Drift stays invisible until somebody looks.

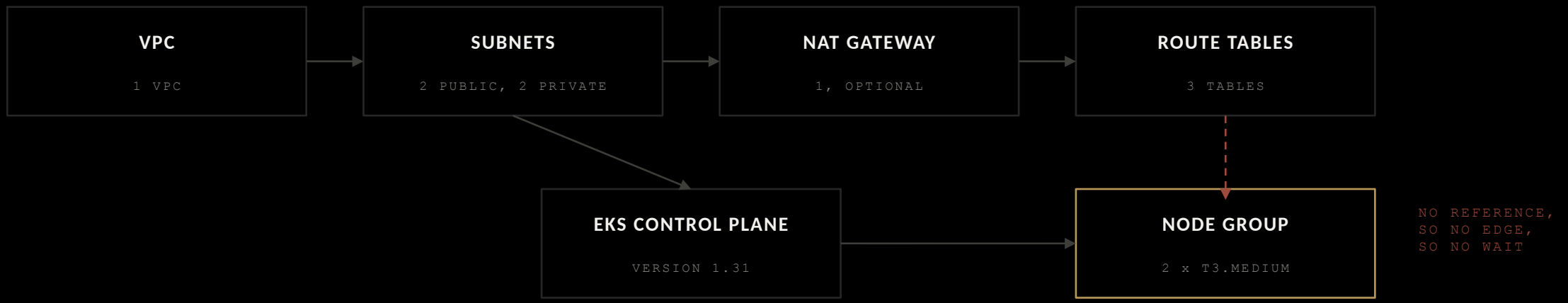
THE PULL MODEL, USED HERE



Nothing outside the cluster can write to it. The audit trail is the commit history, and drift is corrected on the next sync.

The pipeline's last act is a commit, not a deployment.

Terraform had to be told to build the network before the nodes.



Nodes booted into private subnets whose route out did not exist yet. They never joined, and the node group sat in CREATING with an empty health issues list.

```
depends_on = [aws_nat_gateway.main, aws_route_table_association.private]
```

WHAT IT CREATES

VPC CNI, COREDNS, KUBE-PROXY	cluster addons, pinned
EBS CSI + POD IDENTITY	pods cannot reach IMDS, so the role is bound
DEFAULT TAGS	Owner 24BCS10248 on every resource

WHAT IT DOES NOT WARN YOU ABOUT

NO DEFAULT STORAGECLASS	every volume claim waits, silently
NO METRICS SERVER	the autoscaler reads unknown and does nothing
SUBNET ELB TAG	without it a public load balancer never gets an address

One Helm release, with startup order enforced by the runtime.

WHAT THE RELEASE CONTAINS

DEPLOYMENT	backend	2 replicas, anti affinity
DEPLOYMENT	frontend	2 replicas
STATEFULSET	postgres	gp3 volume claim
SERVICE x3	cluster ip	one per workload
INGRESS	nginx	/api and /
HPA x2	cpu 70 percent	backend 2 to 6, frontend 2 to 4
JOB	alembic	upgrade head
SERVICEMONITOR	conditional	rendered only if the CRD exists

WHY THE MIGRATION IS NOT A HOOK



T0 Helm applies every resource at once

T1 Postgres starts. The Job starts with it.

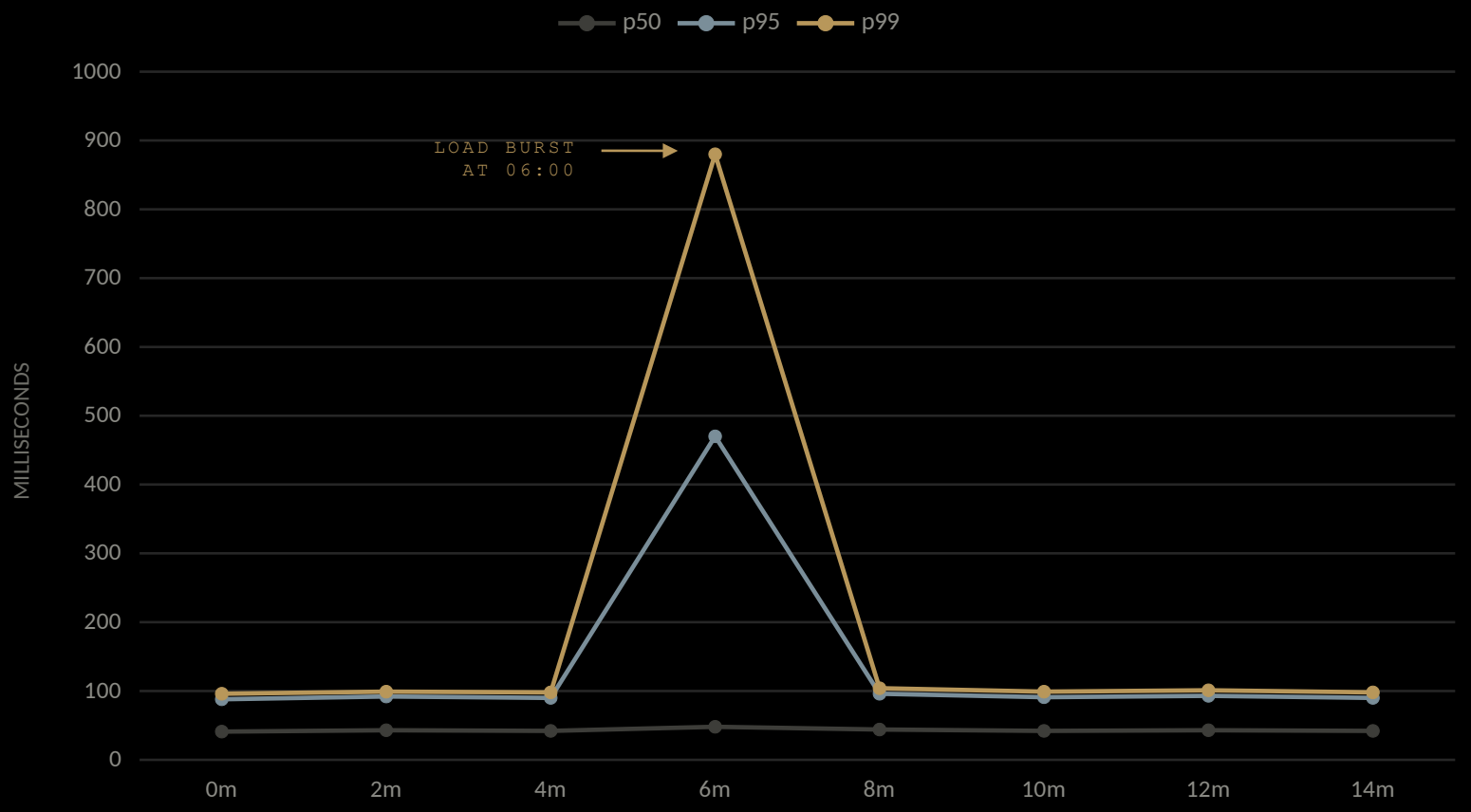
T2 Backend init container waits for the schema

T3 Schema appears. Init exits. Traffic begins.

> A pre install hook runs before every ordinary resource, so hook weights could not help here.

The rule that matters, no traffic against an unmigrated database, is now enforced where it is observable.

An average would have hidden the burst that users actually felt.



THE TAIL

68 ms

mean across the window, looks healthy

THE SAME WINDOW

880 ms

p99, what the slowest users waited

THE SOURCE

cAdvisor

from the kubelet, not from the app

THE ZERO

0 percent

errors as zero, never as no data

Both backend pods scraped and UP. Alert rules live in a PrometheusRule resource, so alerting is versioned with the chart.

Four faults were injected, and each surfaced in a different place.

NO	THE FAULT	WHAT THE CLUSTER DID	WHERE IT SHOWS	EVENT?
01	An image tag that does not exist `kubectl describe pod`	ErrImagePull, restarts forever	*kubelet events*	YES
02	A request for 16 CPUs `kubectl get events --field-selector reason=FailedScheduling`	Pending, never scheduled	*scheduler events*	YES
03	A readiness probe on a missing path `kubectl describe pod, readiness probe failed`	Running, but never ready	*probe failure events*	YES
04	A service selector missing one hyphen `kubectl get endpoints clinicflow-backend`	Ready pod, zero endpoints	*the endpoint list*	NO

Only the fourth is silent. No warning event exists for a selector that matches nothing, so the Service reports itself healthy while serving nobody.

The account was returned to its baseline, verified outside Terraform.

01 DELETE THE HELM RELEASE

Kubernetes owns the load balancer. Terraform does not know it exists.

02 WAIT FOR AWS TO RELEASE IT

Network interfaces drain on AWS time, not on ours.

03 TERRAFORM DESTROY

Only now can the subnets go.

04 SWEEP BY TAG

Every resource carried Owner 24BCS10248, so the sweep is one query.

05 COUNT WHAT REMAINS

Against a baseline taken before the project began.

A cluster that cannot be deleted cleanly was never under control.

BEFORE THE PROJECT, AND AFTER THE SWEEP



Identical. Everything still standing belongs to other projects and was never touched.

Five things that only break once the environment is real.

01	**EXIT ZERO IS NOT EVIDENCE**	An image load reported success for three nodes that did not have the image.	*read state back from the system that owns it*
02	**NOT EMPTY IS NOT SUCCESS**	A readiness poll passed on nginx's own 503 page.	*assert the status code, never the byte count*
03	**A FLOATING VERSION IS A BUG**	A green pipeline turned red with no commit of mine.	*pin the scanner, not just the dependencies*
04	**TERRAFORM INFERS FROM REFERENCES**	A node group needs its route table but never names it.	*if the graph cannot see it, say depends_on*
05	**CLEANUP HAS AN ORDER**	What blocks a destroy is usually what Terraform did not create.	*delete the Kubernetes owned resources first*

EACH ONE FOUND BY READING OUTPUT THAT CONTRADICTED WHAT I EXPECTED

CLINICFLOW

Built, scanned, deployed, measured, and removed.

Application, pipeline, security gate, real cloud infrastructure, Kubernetes, monitoring, and a cleanup I can prove.

WHAT IS IN THE REPOSITORY

`backend/`
fastapi, alembic, 16 tests

`frontend/`
react, vite, nginx

`terraform/`
vpc, eks, node group

`helm/`
one chart, eight objects

`k8s/`
storageclass, argo application

`screenshots/`
evidence for every claim

- THE RULE — DONE
- THE BUILD — DONE
- THE PIPELINE — DONE
- THE CLUSTER — DONE
- THE PROOF — DONE

THANK YOU